# IN5550/IN9550 – Spring 2026
# Obligatory 2: Cross-lingual Word-in-Context

UiO Language Technology Group

**Deadline**: 20 March, 21:59 (Devilry)

## Goals

1. Learn how to finetune pretrained masked language models for downstream tasks.
2. Get practical experience with different loss functions for ordinal classification.
3. Explore zero-shot cross-lingual transfer.

## Recommended reading

1. **Speech and Language Processing** (Daniel Jurafsky and James Martin, 3rd edition)
   - Chapter 10: Masked Language Models
   - Chapter I: Word Senses and WordNet
2. **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding** (Devlin et al., 2019)
3. **Unsupervised Cross-lingual Representation Learning at Scale** (Conneau et al., 2020)
4. **WiC: the Word-in-Context Dataset for Evaluating Context-Sensitive Meaning Representations** (Pilehvar & Camacho-Collados, 2019)
5. **The DURel Annotation Tool: Human and Computational Measurement of Semantic Proximity, Sense Clusters and Semantic Change** (Schlechtweg et al., 2024)

# 1 Introduction

This assignment deals with the **Word-in-Context (WiC)** task: given a target word and two sentences containing it, determine how similar the word's meaning is in the two contexts.

We *highly* recommend using GPU nodes on Fox for development of this assignment (see Appendix A). Your codebase has to be compatible with the available NLPL modules, we recommend these ones in particular:

- `nlpl-transformers/4.55.4-foss-2024a-Python-3.12.3`
- `nlpl-accelerate/1.9.0-foss-2024a-Python-3.12.3`
- `nlpl-nlptools/01-foss-2024a-Python-3.12.3`

Please make sure you read through the entire assignment before you start. Solutions must be submitted via `devilry.ifi.uio.no` by **21:59**, on the **20th of March, 2026**. Please upload a single PDF with details on your experiments and your answers to the questions in the assignment.

Your commented code should be available in a separate **private repository on UiO's hosted GitHub**. Make your repository private but grant the IN5550 staff access to it: `andreku, davisamu, meihant, shenbinq, vladism`. The PDF in Devilry should contain a link to your repository. If you are working on this assignment in a group, please mention all the group members in the report.

Please plan to run your experiments in advance, as this assignment requires time to finetune language models in multiple configurations and obtain the model predictions on a held-out test set.

If you have any questions, please raise an issue in the IN5550 GitHub repository, email us (`in5550-help@ifi.uio.no`), or reach out in the Mattermost chat. Make sure to take advantage of the group sessions.

## 1.1 The Word-in-Context task

The classical WiC task asks a binary question: *does the target word have the same meaning in both sentences?* However, word meaning is rarely black-and-white. The CoMeDi shared task extends WiC to an **ordinal graded** setting, where multiple annotators rate each sentence pair on a scale from 1 to 4, using the DURel relatedness scale by Schlechtweg et al. (2018):

| Label | Meaning |
|---|---|
| 1 | *Unrelated* – the two uses are completely unrelated (homonymy) |
| 2 | *Distantly related* – the uses are only loosely connected (polysemy) |
| 3 | *Closely related* – closely related but distinguishable (context variance) |
| 4 | *Identical* – the uses have the same meaning |

To make this concrete, consider the following examples for the English target word ***arm***:

**Label 4 (Identity):**

(a) *"She broke her **arm** in the fall."*

(b) *"He grabbed my **arm** and pulled me away."*

Both uses refer to a human body part – the meaning is identical.

**Label 3 (Context variance):**

(a) *"She broke her **arm** in the fall."*

(b) *"The **arm** of the Statue of Liberty holds a torch."*

Both uses refer to the same general concept, but the second is a non-prototypical representative (there is some variation in meaning, e.g. the arm of a statue loses the function of the physical arm to be lifted).

**Label 2 (Polysemy):**

(a) *"She broke her **arm** in the fall."*

(b) *"The ship entered an **arm** of the sea."*

Both senses are related via metaphorical extension (physical shape similarity), but they refer to genuinely different things: a body part vs. a narrow body of water.

**Label 1 (Homonymy):**

(a) *"She broke her **arm** in the fall."*

(b) *"The country decided to **arm** its citizens."*

These are completely unrelated meanings: a body part vs. the verb meaning "to supply with weapons."

## 1.2 What makes this dataset special

Two properties of this dataset make it particularly interesting from a modeling perspective:

**Ordinal labels.** The labels 1–4 form a meaningful order, they are not just independent classes. A model that predicts 3 when the true label is 4 has made a smaller mistake than one that predicts 1. Standard classification losses like cross-entropy do not capture this distinction.

**Full annotation distributions.** Each instance comes with the individual judgments of all annotators (typically 2–5), not just the median. For example, an instance might have annotations `[3, 3, 4, 3, 4]`, giving a distribution of $[0.0, 0.0, 0.6, 0.4]$ over labels 1–4. This distribution carries more information than the median alone (3 in this case); it tells us about annotator agreement and semantic ambiguity. A model can be trained to predict this full distribution rather than a single label, also giving us some understanding about the uncertainty of the prediction.

## 1.3 Data format

Each instance in our dataset is a JSON object with the following fields:

```
{
    "sentence_a": "I'd landed in early afternoon, local time.",
    "sentence_b": "Old shopping lists and ... in the faded afternoon.",
    "word": "afternoon",
    "word_indices_a": [20, 29],
    "word_indices_b": [104, 113],
    "labels": [4, 3, 4],
    "median_label": 4
}
```

Here, `word_indices_a` and `word_indices_b` give the character-level start and end positions of the target word in each sentence. The `labels` field contains the individual annotator judgments, and `median_label` is their median. For the blind test set, `median_label` is $-1$ – your goal is to replace these with the correct predictions.

The datasets are available on Fox at: `/fp/projects01/ec403/IN5550/obligatories/2`

The following languages are available. Note that Norwegian has **no training data** – this is intentional and forms the basis for Task B.

| Language | Train | Validation | Test |
|---|---|---|---|
| English (en) | 10 808 | 3 307 | — |
| German (de) | 13 652 | 4 804 | — |
| Chinese (zh) | 20 461 | 5 772 | — |
| Russian (ru) | 12 698 | 3 411 | — |
| Spanish (es) | 9 339 | 2 118 | — |
| Norwegian (no) | – | 6 495 | — |
| Unknown test language | — | — | ??? |

## 1.4 Evaluation

Following the CoMeDi shared task, our primary evaluation metric is **Krippendorff's** $\alpha$ with ordinal distance (Krippendorff, 2019). This metric was originally designed to measure inter-annotator agreement, but it can also be used to compare model predictions against gold labels. Unlike accuracy, Krippendorff's $\alpha$ accounts for the ordinal structure of the labels: predicting 3 instead of 4 is penalized less than predicting 1 instead of 4. The metric ranges from $-1$ (systematic disagreement) through 0 (chance agreement) to 1 (perfect agreement).

You should also report **accuracy** as a secondary metric, i.e., the proportion of instances where the predicted median label exactly matches the gold median label.

Krippendorff's $\alpha$ on the unknown test set will serve as the main metric for the leaderboard.

## 1.5 Provided code

We provide an evaluation script to help you get started with this task. You can find it in the IN5550 GitHub repository.

`evaluation.py`

This script can be called as `python3 evaluation.py pred.jsonl gold.jsonl` as it will print out the Krippendorff's $\alpha$ and accuracy. You can also import the `evaluate_batch(predicted_label: List[int], gold_label: List[int]) -> dict` method in any of your scripts for computing the evaluation metrics. Note that Krippendorff's $\alpha$ should only be computed on a "batch" of many samples (ideally the whole data split) to give a meaningful score.

# 2 Assignment

The assignment is divided into three parts (plus an additional task for PhD students). Read through all parts before starting.

## 2.1 Task A: Modeling (6 points)

Your first goal is to train and compare different approaches for the WiC task, **using only the English training and validation data**. For each experiment below, train models for at least 10 epochs and report both training loss curves and validation Krippendorff's $\alpha$ per epoch.

### A1: Static embedding baseline (1 point)

Before diving into finetuning, build a simple baseline using **pretrained static word embeddings**. The idea is straightforward: look up the embedding of the target word and of the surrounding $N$ words in each sentence, average those embeddings for each sentence, and pass those to a simple MLP classifier.

This subtask should not take much time – the point is to establish a simple baseline and to illustrate *why* contextualized representations matter for this task.

- Report the Krippendorff's $\alpha$ and accuracy on the English validation set.
- Why do static embeddings struggle with this task? What fundamental limitation do they have that contextualized models address?

### A2: Language model comparison (1 point)

Compare finetuning the original BERT (`bert-base-cased`; Devlin et al., 2019) against a modern multilingual model such as mmBERT (`jhu-clsp/mmBERT-base`; Marone et al., 2025). Please see Appendix B for some useful tips about loading HuggingFace models on Fox.

- Create plots of the training loss and Krippendorff's $\alpha$ curves per epoch on the English validation set for both models.
- Analyze how well each model converges and compare their final performance.
- What differences do you observe? Can you explain them based on the differences between the two models?

### A3: Output pooling (2 points)

The standard BERT finetuning approach concatenates the two sentences together (with a special token as a separator), and takes the representation of the `[CLS]` / beginning-of-sequence token, passing it through a classification head. However, for WiC, we can also **pool the representations of the two target word tokens** (using the provided character indices to identify them after tokenization) and use these as input to the classifier.

Compare these two pooling strategies:

1. **BOS pooling**: use the representation of the beginning-of-sequence token (the canonical approach).
2. **Target-word pooling**: extract the contextualized embeddings of the target word in each of the two sentences (e.g., by averaging the subword embeddings of the target word), concatenate both of them, and use this as input to an MLP classification head. To implement this method, please look at Appendix D first.

Which strategy performs better? Why might explicitly extracting the target-word representations help (or not help) the model?

### A4: Loss functions (2 points)

The ordinal nature of the labels and the availability of full annotation distributions open up interesting possibilities for the loss function. Compare the following four loss functions:

1. **Mean Squared Error (MSE) loss on median labels**: treat the task as regression by predicting a single scalar value and using mean squared error against the median label. At inference time, round the prediction to the nearest integer label (1–4).
2. **Cross-entropy loss on median labels**: treat the task as standard 4-class classification using the median label as the target.
3. **Squared EMD loss on median labels**: use the squared Earth Mover's Distance (Hou et al., 2017) between the predicted class distribution and a one-hot encoding of the median label. This respects the ordinal structure – the loss penalizes predictions that are ordinally farther from the truth. You can find a detailed description of this loss in Appendix C.
4. **Squared EMD loss on the full distribution**: instead of comparing against a one-hot median, use the full annotator distribution as the target (e.g., `[3,3,4,3,4]` $\rightarrow [0, 0, 0.6, 0.4]$). This trains the model to capture the actual distribution of human judgments, preserving information about disagreement and uncertainty. Again, please refer to the proper description

of this loss in Appendix C.

- Which loss function achieves the best performance? Does the ranking match your expectations?
- Analyze the advantages and disadvantages of each loss. In particular, discuss: why might MSE be suboptimal despite also encoding order? What does the EMD loss gain from respecting the ordinal structure? What additional information does the full distribution provide compared to the median?

## 2.2 Task B: Cross-lingual transfer (4 points)

Norwegian has no WiC training data in our dataset. The goal of this part is to build the best possible WiC model for Norwegian by transferring knowledge from other languages.

### B1: Transfer based on language similarity (2 points)

Using your best configuration from Task A (model, pooling, and loss), train four models on different training sets:

1. English,
2. Chinese,
3. Spanish,
4. German.

- For each model, report the Krippendorff's $\alpha$ on the validation sets of **all available languages** (including Norwegian).
- Analyze how the performance on Norwegian changes as we change the language we train on. Does training on Chinese help Norwegian? What about German? Does the Norwegian performance depend only on language similarity or are there other important factors to consider?

### B2: Your own language selection (2 points)

Design your own multilingual training set by selecting from the available training languages. Your goal is to maximize performance on the Norwegian validation set.

- Motivate your choice of languages and describe it in the report.
- Compare the Norwegian performance to your best model from B1 and analyze the differences.

- Did your language selection strategy work as expected? What would you try differently next time?

## 2.3 Task C: Test predictions and evaluation (2 points)

Using your best overall model (from any of the previous tasks), produce predictions on the unknown test test. We will give you access to the blind (without any labels) dataset three days before the deadline. The specific language of that dataset is a secret, but it will one of the mainland Scandinavian languages.

- Write a **prediction script** (`predict.py`) that takes a trained model and a blind dataset, producing the prediction file. The script should be runnable by us, make sure you make your model accessible to us on Fox. Please include clear usage instructions in a `README`.
- Push the predictions to your GitHub repository. The prediction file should follow the `.jsonl` format of the other data splits and it should be compatible with the provided evaluation script.
- In your report, briefly describe your best model configuration and why you chose it.

## 2.4 Task D: Enriching Wikipedia (only for PhD students enrolled in IN9550)

At this point, you are a thriving researcher, already having a good understanding of using deep learning methods for NLP. That's why we want you to contribute a small part of your knowledge to the world's largest open encyclopedia - that is, Wikipedia. Most Wikipedias (except for English) have significant gaps in the coverage of NLP and ML topics. You can help to at least partially address this issue by using what you have already learned in the IN9550 course (and elsewhere).

Specifically, your task is to enrich the Wikipedia in your native or near-native language with the content related to this obligatory assignment. The suggested topics are:

- Contextual embeddings;
- Transformer architecture;
- Masked language models;
- Self-Attention;
- Finetuning of large language models;
- Representation learning;
- Generative language models

- large language models for Norwegian (if you choose to contribute to Norwegian Wikipedia);
- ...

You should identify one existing gap in the Wikipedia of your choice and address it by either 1) creating a new article on one of the topics above, or 2) adding at least one paragraph (no less than 200 words) to one of the existing articles.

You contributions do not have to cover the chosen topic in its entirety (for example, you newly created article can be a stub), but they should be valuable: that is, they should add new information.

It goes without saying that the facts you contribute in your text should be supported by references, following the best academic practices and Wikipedia guidelines. If this is your first time contributing to Wikipedia, please get acquainted with these tutorials:

1. https://en.wikipedia.org/wiki/Help:Introduction
2. https://en.wikipedia.org/wiki/Wikipedia:Contributing_to_Wikipedia

In the assignment report, you must provide your Wikipedia username and the link to the revision you made. The revision should be made no less than 24 hours before the submission deadline, so that we see that it is not immediately removed by the Wikipedia editors.

## 3 Submission details

You are to submit a written report (roughly 6 pages) to Devilry, which provides details on your experiments and addresses the questions posed in the assignment. Your code should be available in your UiO GitHub repository (private but accessible by IN5550 staff).

Your code must be in Python (`*.py` scripts, **not** Jupyter notebooks), and one should be able to run it directly on the provided data. Please use Python modules included in our NLPL Virtual Laboratory (unless you have very compelling reasons to do otherwise).

Good luck!

## References

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. **Unsupervised cross-lingual representation**

**learning at scale**. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 8440–8451, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.747.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. **BERT: Pre-training of deep bidirectional transformers for language understanding**. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423.

Le Hou, Chen-Ping Yu, and Dimitris Samaras. **Squared earth mover's distance-based loss for training deep neural networks**, 2017.

Klaus Krippendorff. **Content Analysis: An Introduction to Its Methodology (second edition)**. Sage Publications, 2019.

Marc Marone, Orion Weller, William Fleshman, Eugene Yang, Dawn Lawrie, and Benjamin Van Durme. **mmBERT: A modern multilingual encoder with annealed language learning**, 2025.

Mohammad Taher Pilehvar and Jose Camacho-Collados. **WiC: the word-in-context dataset for evaluating context-sensitive meaning representations**. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 1267–1273, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1128.

Dominik Schlechtweg, Sabine Schulte im Walde, and Stefanie Eckmann. **Diachronic usage relatedness (DURel): A framework for the annotation of lexical semantic change**. In Marilyn Walker, Heng Ji, and Amanda Stent (eds.), *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pp. 169–174, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-2027.

Dominik Schlechtweg, Shafqat Mumtaz Virk, Pauline Sander, Emma Sköldberg, Lukas Theuer Linke, Tuo Zhang, Nina Tahmasebi, Jonas Kuhn, and Sabine Schulte Im Walde. **The DURel annotation tool: Human and computational measurement of semantic proximity, sense clusters and semantic change**. In Nikolaos Aletras and Orphee De Clercq (eds.), *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pp. 137–149, St.

Julians, Malta, March 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.eacl-demo.15.

# A Using GPUs on Fox

We strongly recommend to train the language models on a GPU using the Fox cluster. It will save you a ton of time :) You can follow the GPU SLURM script from our GitHub repository. Then, in your Python script, the simplest way of running your code on a GPU is to set the default device to CUDA using `torch.set_default_device('cuda')`. Now every tensor will be automatically allocated on the GPU memory. You can also manually move any tensor to the CPU memory by running `tensor.cpu()` and vice-versa by running `tensor.cuda()`. The same works for any PyTorch module (such as your model).

However, keep in mind that there is a large demand for GPU resources on Fox. So, when debugging, it is much better to do that on a CPU node so as to not wait in a long queue. It might also be a good idea to use a smaller language model for debugging (such as mmBERT-small, available at `/fp/projects01/ec403/hf_models/mmBERT-small`). For the same reason, try to limit for how long you train the models, there is no benefit in running them on more than 10 epochs – much less usually works even better with good hyperparameters.

# B Loading HuggingFace models on Fox

The language models are already downloaded on Fox, please save your user space and load them from `/fp/projects01/ec403/hf_models/`. For example, to load and run the mmBERT-base model:

```
import torch
from transformers import AutoTokenizer, AutoModelForMaskedLM
torch.set_default_device('cuda')

# Import model
tokenizer = AutoTokenizer.from_pretrained(
    "/fp/projects01/ec403/hf_models/mmBERT-base"
)
model = AutoModelForMaskedLM.from_pretrained(
    "/fp/projects01/ec403/hf_models/mmBERT-base",
    device_map="cuda",  # this uses accelarate to initialize the parameters directly on GPU
    torch_dtype="bfloat16"  # speeds-up computation and uses less memory, but risks numerical issues
```

```
)

# Tokenize text (with a mask token inside)
input_text = tokenizer(
    f"Jeg elsker å spise{tokenizer.mask_token}.",
    return_tensors="pt"
)

# Inference
output_p = model(**input_text)

# Unmask the text
output_text = torch.where(
    input_text.input_ids == tokenizer.mask_token_id,
    output_p.logits.argmax(-1),
    input_text.input_ids
)

# Decoding
print(tokenizer.decode(output_text[0].tolist()))
```

# C Earth Mover's distance (EMD) loss

The squared Earth Mover's Distance (EMD) loss (Hou et al., 2017) works by comparing **cumulative distribution functions** (CDFs) of the predicted and target distributions. Here is how to compute it step by step:

1. **Obtain the predicted distribution.** Your model outputs $K = 4$ raw logits $z_1, \ldots, z_K$. Apply the softmax function to obtain a valid probability distribution:
$$\hat{p}_k = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \quad k = 1, \ldots, K$$

2. **Construct the target distribution.** For the *median* variant, this is a one-hot vector: if the median label is $c$, then $q_c = 1$ and $q_k = 0$ for $k \neq c$. For the *full distribution* variant, normalize the annotator label counts: e.g., labels `[3, 3, 4, 3, 4]` give $\mathbf{q} = [0,\ 0,\ 0.6,\ 0.4]$.

3. **Compute the CDFs** of both distributions by taking the cumulative sum:
$$\mathrm{CDF}_{\mathrm{pred}}(k) = \sum_{j=1}^{k} \hat{p}_j, \qquad \mathrm{CDF}_{\mathrm{target}}(k) = \sum_{j=1}^{k} q_j$$

4. **Compute the squared EMD loss** as the mean of squared differences

between the two CDFs:

$$\mathcal{L}_{\text{EMD}^2} = \frac{1}{K}\sum_{k=1}^{K}\left(\text{CDF}_{\text{pred}}(k) - \text{CDF}_{\text{target}}(k)\right)^2$$

The key insight is that by comparing CDFs rather than probability mass functions, the loss implicitly encodes the ordinal distance between classes. For example, if the true label is 4, predicting class 3 shifts only the last CDF value, while predicting class 1 shifts all CDF values – resulting in a much larger loss. This is exactly the property we want for ordinal data.

# D Getting offsets of subword tokens

To properly implement the target-word pooling, you will need to find which tokens form that word. `transformers` offer a useful argument when tokenizing that makes this much easier to accomplish.

As an example, let's use a sentence from the English dataset *He's never broke his word to them yet.*, where the lemma to look for is *word* that appears as a substring between character positions 21 and 25 (this is from the `"word_indices_a"` field in the dataset). Now we can use the argument `"return_offsets_mapping=True"` when tokenizing to get this:

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(
    "jhu-clsp/mmBERT-base"
)
encoding = tokenizer(
    "He's never broke his word to them yet.",
    add_special_tokens=False,
    return_offsets_mapping=True
)
print(encoding["offset_mapping"])
# this should return [(0, 2), (2, 3), (3, 4), (4, 10), (10, 16), (16, 20), (20, 25), (25, 28), (28,
```

This list corresponds to the substring positions of each subword (so here, we can see that the first subword is `He`, then we have `'` and `s`; the next subword is `_never` – including the leading space!).

Looking for our target word, we see that it has been tokenized as a single token with a leading space, it's the 7th token with character offsets `(20, 25)`.